JAVA PRACTICAL TEST

10 Programming Questions • Loops to File Handling

Topics covered: Loops, arrays, OOP, ArrayList, HashSet, HashMap, multithreading, and file handling.

1. Number Pattern and Loops [EASY]

Write a Java program to print all Armstrong numbers between 1 and 1000 using loops.

• Use loops; do not use a built-in method to check Armstrong numbers.

Example output: 1, 153, 370, 371, 407

2. Array Operations [EASY]

Given an integer array, find the second-largest distinct element without using Arrays.sort().

• Input: [12, 45, 7, 45, 32, 18]

• Output: Second largest: 32

• Handle duplicate values and the case where no second-largest distinct element exists.

3. OOP – Inheritance and Polymorphism [MEDIUM]

Create a class hierarchy for an employee salary calculation system.

• Create parent class Employee with employee ID, name, and basic salary.

• Create FullTimeEmployee and PartTimeEmployee subclasses.

• Override calculateSalary(). Full-time employees receive a 20% bonus; part-time salary = hours worked × hourly rate.

• Display employee details and final salary using runtime polymorphism.

4. ArrayList – Student Management [MEDIUM]

Create a student management program using ArrayList. Each student has a roll number, name, and mark.

• Add at least five students.

• Display students scoring above 75.

• Search by roll number, update a mark, and remove a student by roll number.

• Display students sorted by marks without using a database.

5. HashSet – Remove Duplicates [MEDIUM]

Remove duplicate integers from an array using HashSet.

• Input: [10, 20, 10, 30, 20, 40, 50, 30]

• Display unique values and the number of unique elements.

• Check whether the set contains 40.

• Explain why output order is not guaranteed.

Expected unique values: 10, 20, 30, 40, 50 (order may vary).

6. HashMap – Word Frequency Counter [MEDIUM]

Count how many times each word occurs in a sentence using HashMap.

• Input: java is easy and java is powerful

• Ignore case differences and store each word with its count.

• Find the most frequently occurring word and handle an empty sentence.

Expected counts: java=2, is=2, easy=1, and=1, powerful=1.

7. Multithreading – Ticket Booking [HARD]

Create a ticket booking simulation using multithreading.

• TicketCounter starts with 10 tickets; create three customer threads.

• Each customer attempts to book 4 tickets.

• Use synchronization to prevent race conditions.

• Display successful bookings and remaining tickets; tickets must never become negative.

8. Multithreading – Producer and Consumer [HARD]

Implement a basic producer-consumer program using two threads.

• Producer generates numbers 1 through 5; consumer reads and displays each number.

• Use wait() and notify() or notifyAll() for coordination.

• Ensure each number is consumed only after it is produced.

• Terminate both threads correctly after processing all numbers.

9. File Handling – Student Records [HARD]

Create a Java program to save and retrieve student records from a text file.

• Accept student ID, name, and marks; write at least five records to students.txt.

• Read and display all records, then find the student with the highest marks.

• Append a new student without deleting existing records.

• Handle IOException appropriately.

10. Mini Project – Employee Record Manager [HARD]

Build a menu-driven Java application combining OOP, collections, and file handling.

• Create Employee with ID, name, department, and salary.

• Use ArrayList to store employees, HashMap for lookup by ID, and HashSet to enforce unique IDs.

• Implement add, search, update, delete, and display operations.

• Save records to a file and load them when the program starts.

• Validate user input and handle exceptions.

Sample menu: 1. Add Employee 2. Search Employee 3. Update Salary 4. Delete Employee 5. Display All 6. Save and Exit

SUGGESTED TEST RULES

Duration	3 hours
Total marks	100
Questions 1–2	5 marks each
Questions 3–9	10 marks each
Question 10	20 marks

Evaluation criteria: Logic and correctness, appropriate use of Java concepts, exception handling, code readability, and successful execution.

Trainer tip: Have students complete Questions 1–6 first, then attempt multithreading and file handling. Use Question 10 as the final challenge.